

Software-Engineered Chest X-Ray Analysis and Explanation System

Assignment II — Engineering Implementation and Quality Assurance

Domain	Healthcare / medical imaging
Engineering focus	Modular ML software, automated QA and deployment controls
Group	Group 89

0. Group Details & Contribution

Group No: 89

Implementation ownership was divided across requirements, data quality, ML pipeline construction, architecture, validation and report integration. Each contribution is tied to a concrete engineering responsibility within the delivered system.

#	BITS ID	Name	Contribution	Weight
1	2024ac05672@wilp.bits-pilani.ac.in	Ananda Vadivel D	Requirements formulation, GR4ML view preparation, quality requirement justification	25%
2	2024ac05968@wilp.bits-pilani.ac.in	Adurti Sai Venkatesh	Dataset preparation, exploratory data analysis, visualization support, data quality validation	25%
3	2024ac05653@wilp.bits-pilani.ac.in	D Mallikarjuna Reddy	ML pipeline implementation, model comparison, final evaluation, report integration	25%
4	2024ac05055@wilp.bits-pilani.ac.in	Krupashankar Subramani	System architecture, architectural pattern mapping, model registry artifacts, inference and application validation	25%

Engineering Requirements Traceability

The traceability table connects each implemented engineering capability to its concrete mechanism, verification evidence and detailed report location. It provides a compact system-level index without duplicating the technical analysis in later sections.

Engineering capability	Implemented mechanism	Verification evidence	Report location
Modular design	Separated data, model, workflow, inference and telemetry packages.	Topology and service-contract audits	Sections 3-4
Research-production parity	Notebook probability path paired with ComputerVisionService.	10/10 contracts; parity at 1e-6	Section 5
Operational logging	INFO, WARNING and ERROR events in three critical services.	Six structured-logging tests	Section 6
Static code quality	Pinned Black, isort and Ruff with transactional remediation.	Zero findings; compile and regression pass	Section 7
Typed inference API	Strict schemas, status codes, error contracts and versions.	12 paths and 12 OpenAPI operations	Section 12; App. A
Multi-layer testing	Unit, data, model, security, API and deployment tests.	81 repository-wide tests pass	Section 9
ML behavior contracts	Small-batch optimization and frozen inference parity.	Loss, shape, range and parity assertions	Sections 8.2, 11
Quality measurement	Data integrity plus ROC-AUC, AP and F1 metrics.	Versioned data/model metric artifacts	Sections 10-11
Production experiment	Deterministic 10% non-interfering shadow route.	Routing and isolation tests	Section 14
Security control	Media, signature, size, integrity and dimension validation.	Six adversarial upload tests	Section 13

Table 1. Engineering capability, implementation and verification traceability.

Table of Contents

Engineering Requirements Traceability

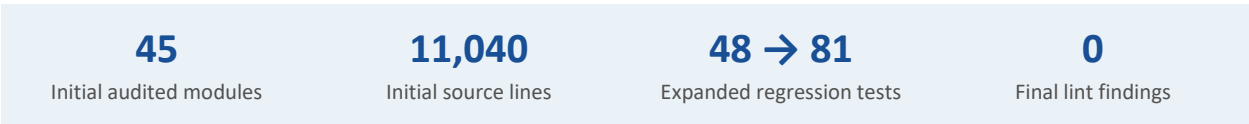
1. Executive Summary
 2. Engineering Context and System Scope
 3. Architecture and Responsibility Boundaries
 4. Modularity and Object-Oriented Contracts
 5. Research-to-Production Refactoring
 6. Exception Handling and Structured Logging
 7. Code Quality and Static Analysis
 8. Production Data and Training Components
 9. Test Strategy and Regression Evidence
 10. Data Quality and Drift Controls
 11. Model Quality and Inference Integrity
 12. FastAPI Contract and Application Integration
 13. Security Engineering
 14. Shadow Deployment Strategy
 15. Packaging, Reproducibility and Deployment Readiness
 16. Evidence Integrity and Final Readiness
 17. Technical Limitations and Next Engineering Iterations
 18. Conclusion
- Appendix A. Endpoint Catalogue
- Appendix B. Notebook and Artifact Traceability
- Appendix C. Evidence Registry
- Appendix D. Selected Implementation Excerpts
- References

1. Executive Summary

The Software-Engineered Chest X-Ray Analysis and Explanation System converts an interactive machine-learning workflow into a modular, testable and deployable software system. The implementation separates data access, model construction, image validation, classification, visual explanation, grounded language generation, prediction persistence, operational metrics, API schemas and user-interface integration into explicit components with validated contracts.

Engineering evidence was generated through static repository audits, public-symbol inspection, exception-path analysis, structured logging tests, deterministic data checks, frozen-model inference parity, upload-security tests, OpenAPI schema tests, shadow-deployment policy tests, packaging contracts and a repository-wide regression suite. All thirteen final readiness gates passed.

The implementation unifies modular service boundaries, research-to-production parity, structured logging, typed API contracts and automated quality gates. Model training and inference behavior, data integrity, security controls and deployment experimentation are verified through executable tests and versioned evidence artifacts.



Verified outcome

- The service layer exposes stable object-oriented boundaries without changing the fourteen-label inference contract.
- Research and production probability vectors match exactly within an absolute tolerance of 1×10^{-6} .
- The frozen test partition contains 22,433 images; threshold recalibration and test inference reruns were explicitly excluded from the final evidence extraction.
- The deployment package defines API and UI services, health checks, pinned dependencies and external model/data mounts without embedding large runtime artifacts.
- A SHA-256 evidence manifest binds 52 evidence files and 80 repository files to the final readiness decision.

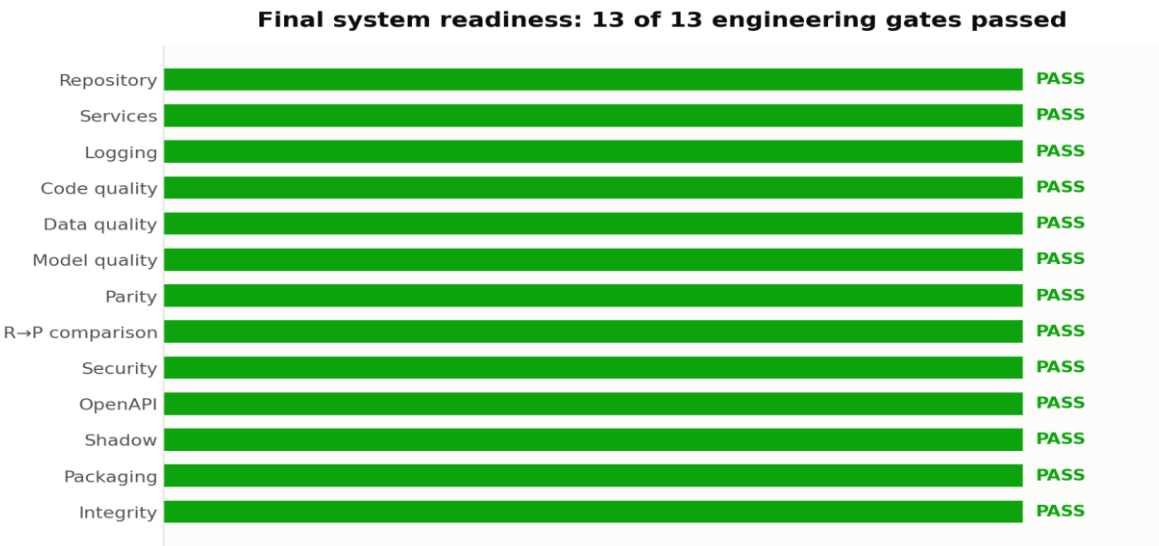


Figure 1. Final readiness gate summary generated from the validated evidence registry.

2. Engineering Context and System Scope

The system accepts a chest X-ray image, validates its media and dimensional contract, produces fourteen ordered finding probabilities, identifies threshold crossings, generates Grad-CAM visual evidence, serializes grounded findings for language generation, applies deterministic output guardrails and exposes the result through typed API and UI workflows. The software scope includes both the ML execution path and the controls required to operate that path predictably.

2.1 Functional execution path

1. Validate the declared media type, decoded format, byte size, pixel dimensions and image integrity.
2. Normalize and preprocess the image under the frozen model input contract.
3. Execute ResNet-18 inference and preserve the configured fourteen-label order and per-label thresholds.
4. Generate visual evidence only for threshold-crossed findings using the configured Grad-CAM target layer.
5. Persist the prediction record and attach explainability and grounded-language outputs through controlled upserts.
6. Return strict response schemas with request metadata, component versions, latency and controlled errors.

2.2 Software-quality objectives

Each software-engineering practice below was selected because it maps to a specific quality risk in ML systems rather than to general application code: modularity keeps the data, model and serving concerns independently testable; observability and reproducibility let a prediction be traced back to the exact model version and input contract that produced it; and automated QA gives an objective, repeatable signal in place of ad-hoc notebook checks.

Quality attribute	Implemented control	Verification mechanism
Modularity	Service, workflow, schema and configuration boundaries	AST topology and public-contract audit
Correctness	Frozen label order, thresholds and tensor contracts	Inference parity and behavior tests
Observability	Structured lifecycle, validation, inference and guardrail events	Six behavior tests plus event inventory
Security	Content-aware upload validation and controlled error responses	Six adversarial upload tests
Reproducibility	Pinned dependencies, configuration artifacts and SHA-256 evidence	Packaging and integrity contracts
Deployability	Separate API/UI images, health checks and external artifact mounts	Eleven deployment contract tests

3. Architecture and Responsibility Boundaries

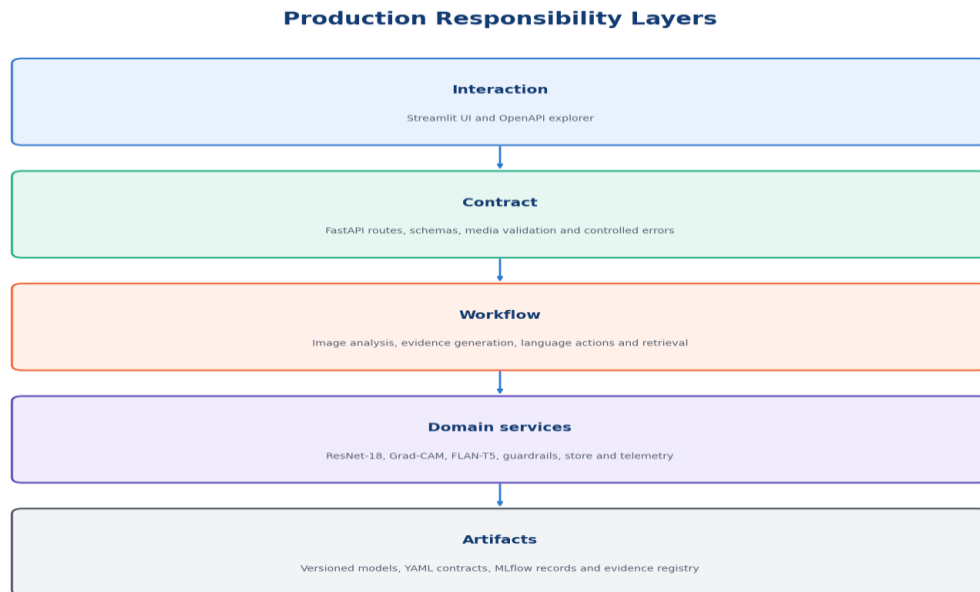


Figure 2. Layered production architecture and responsibility separation.

The architecture uses dependency injection through a ServiceContainer. Construction is isolated in the factory, request handlers depend on declared services, and workflow classes coordinate multiple services without owning model construction. This keeps HTTP concerns, ML execution, persistence, telemetry and schema serialization independently testable.

3.1 Runtime component map

Layer	Primary modules	Owned responsibility
Interaction	ui.app, ui.components, ui.api_client	Session state, upload workflow, evidence rendering, retrieval and metrics views
HTTP contract	api.main, api.routes.*, api.schemas.*	Routing, strict validation, response types, exception translation and OpenAPI
Orchestration	image_workflow, language_workflow, complete_workflow	Sequencing, metadata propagation, record updates and service invocation metrics
Domain services	image, computer_vision, gradcam, grounding, language, guardrail	Validation, inference, explanation, serialization and controlled generation
State and operations	prediction_store, operational_metrics	Bounded in-memory retention, thread-safe updates and operational snapshots
Configuration	configs/*.yaml, *.json	Finding order, thresholds, paths, prompts, limits and response templates

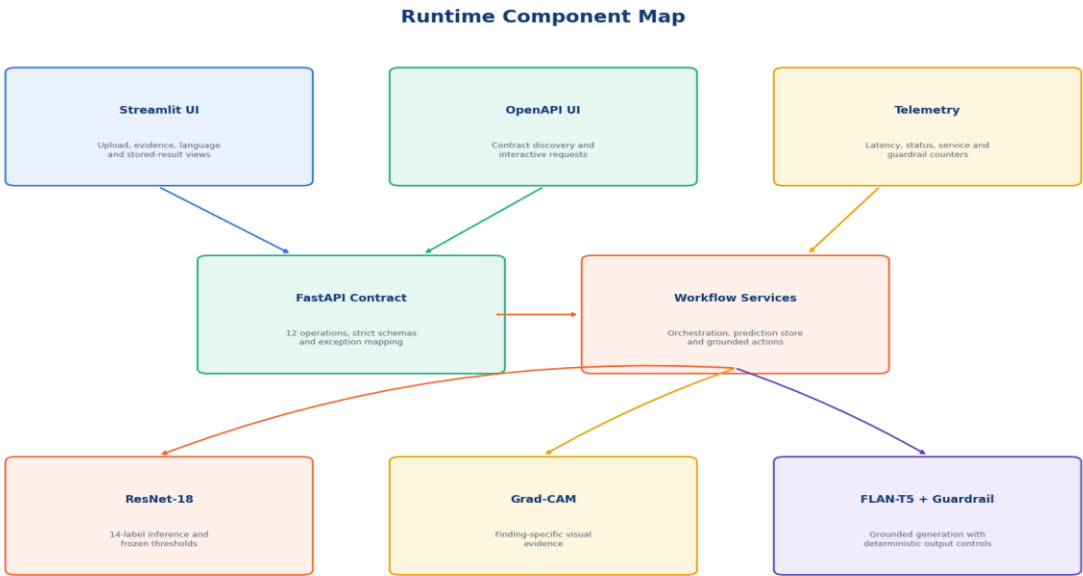


Figure 3. System component architecture as exercised by the application workflows.

4. Modularity and Object-Oriented Contracts

A static AST audit parsed all 45 initial Python modules without syntax failure. The audited baseline contained 61 classes, 74 functions and 77 dependency references across 11,040 source lines. The service-contract audit then validated sixteen requested modules, twenty-seven classes, forty-two public methods and fourteen public functions with no missing or invalid module.

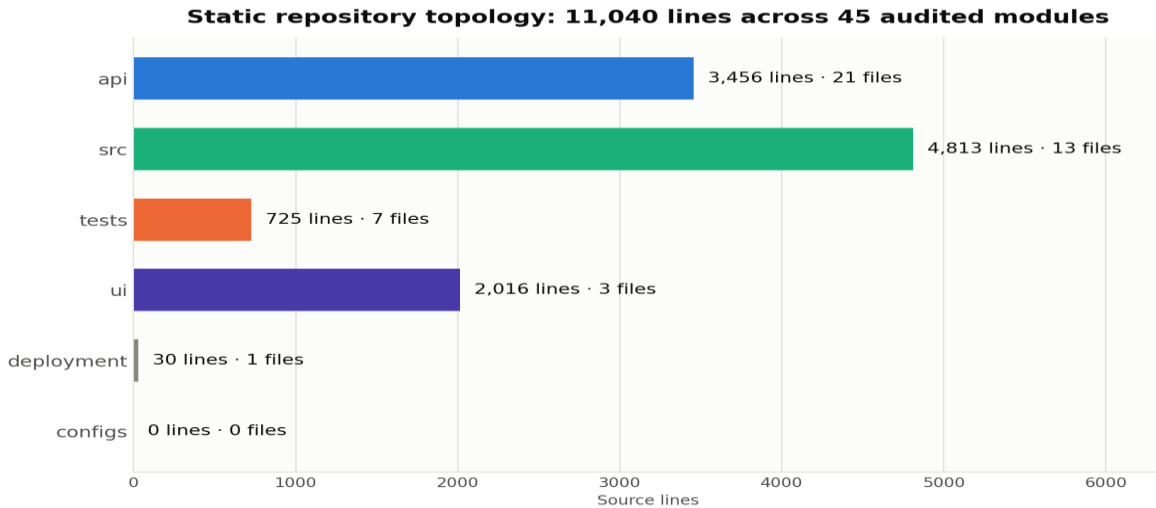


Figure 4. Audited repository topology before the additional QA modules were introduced.

4.1 Core service contracts

Component	State and synchronization	Public contract
ImageValidationService	Validated settings; no raw content retained	validate_and_decode → ValidatedImage
ComputerVisionService	Model, thresholds, device and inference lock	preprocess; predict; is_ready; confidence_category
GradCAMService	Target layer, device, dtype and generation lock	generate; generate_for_crossed_findings
GroundedInputSerializer	Model version, task prefixes and limitation contract	normalize_finding; serialize
GroundedLanguageModelService	Tokenizer, model, device and inference lock	generate; generate_many
DeterministicLanguageGuardrail	Required limitation and review strings	audit; apply; build_fallback
PredictionStoreService	Thread-safe bounded record store and retention	create; get; attach; upsert; snapshot
OperationalMetricsService	Thread-safe counters and bounded latency samples	record_*; snapshot

4.2 Design properties

- Encapsulation: model and tokenizer lifecycle state remains private to the service that owns it.
- Single-responsibility boundaries: validation, inference, explainability, serialization, guardrails and persistence are separate modules.
- Dependency inversion: workflow services receive collaborators and metadata builders through constructors.

- Concurrency control: inference, generation, metrics and record operations use explicit locks around shared mutable state.
- Schema stability: dataclass result objects and Pydantic response models preserve typed boundaries across layers.

5. Research-to-Production Refactoring

The production implementation preserves the trained model’s numerical behavior while relocating notebook-controlled state into reusable software components. The comparison uses a frozen reference probability path and validates both structural contracts and numerical parity.

Dimension	Research implementation	Production implementation
Execution boundary	Notebook-controlled execution state	Reusable ComputerVisionService instance
Model lifecycle	Construction and restoration in cells	Service-owned construction, restoration and readiness
Configuration	Notebook variables and metadata	Validated ServiceSettings dependency
Preprocessing	Transformation sequence executed directly	Reusable preprocess method with fixed tensor contract
Concurrency	Single interactive flow	Inference lock protects shared model state
Failure handling	Notebook exception propagation	ModelNotReadyError and ServiceExecutionError
Observability	Cell output and evaluation artifacts	Structured lifecycle and inference events
Output	Fourteen sigmoid probabilities	ClassificationResult with ordered finding evidence
Verification	Frozen reference probability path	Automated numerical parity and contract tests

10/10	14	1×10^{-6}	0.0000000000
Production contracts	Ordered labels	Absolute tolerance	Maximum difference

The zero observed difference demonstrates that the refactoring changed ownership and execution structure without changing the output vector. The test also verifies that the frozen checkpoint was not modified.

6. Exception Handling and Structured Logging

Exception control begins with domain-specific `ServiceError` subclasses and ends with typed API error responses. Invalid images, unsupported media, excessive payloads, unavailable models, missing predictions, grounding-contract violations and service execution failures are translated without exposing internal exception text or sensitive request content.

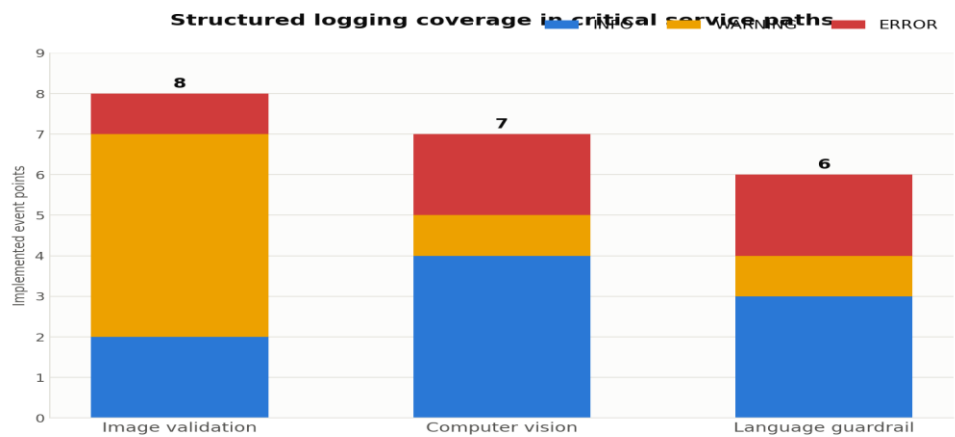


Figure 5. Implemented log-event points by severity and critical module.

6.1 Exception boundary

Failure class	Detection location	External behavior
UnsupportedMediaTypeError	Declared type and decoded format validation	Controlled 4xx response with supported-type details
InvalidImageError	Empty, corrupt or dimensionally invalid image	Controlled image-validation error
UploadTooLargeError	Maximum upload-byte boundary	Controlled size error without reading further workflow state
ModelNotReadyError	Model load or inference readiness check	Component readiness failure
PredictionNotFoundError	Prediction-store lookup	Controlled missing-record response
GroundingContractError	Finding serialization or generation precondition	Controlled grounded-language failure
ServiceExecutionError	Unexpected inference/generation execution	Sanitized service failure with request metadata

6.2 Sensitive-content exclusion

Logging behavior tests verify event semantics and exclusion boundaries. Logs do not contain raw image bytes, uploaded filenames, checkpoint paths, tensor values, probability arrays, generated text, user questions, serialized prompts or finding evidence. Six dedicated tests passed in 2.98 seconds.

The implemented log statements use stable event names and structured scalar metadata so that operational queries do not depend on free-form message parsing.

7. Code Quality and Static Analysis

A pinned development toolchain was defined in requirements.dev.txt and pyproject.toml: pytest 8.3.5, Black 24.10.0, isort 5.13.2 and Ruff 0.9.10 targeting Python 3.11 across src, api, ui and tests. The baseline was captured without modifying source files, followed by transactional remediation with a snapshot and rollback boundary.

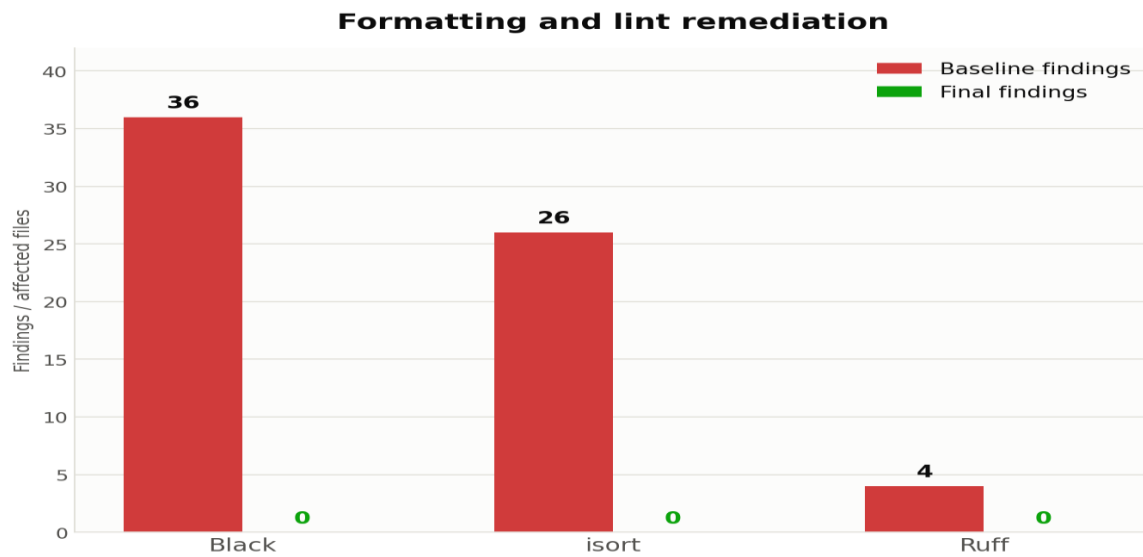


Figure 6. Baseline findings and final zero-finding state.

7.1 Transactional remediation

1. Snapshot all 45 in-scope Python files before mutation.
2. Run the complete pre-remediation regression suite and require a zero return code.
3. Apply Ruff fixes, isort import normalization and Black formatting in a deterministic order.
4. Compile the repository and re-audit public symbols to detect structural contract changes.
5. Run the post-remediation suite; restore the snapshot automatically on any gate failure.

Gate	Baseline	Final	State
Black	36 affected files	0	PASS
isort	26 affected files	0	PASS
Ruff	4 findings	0	PASS
compileall	Not a finding count	Return code 0	PASS
Public symbols	Captured	Unchanged	PASS

8. Production Data and Training Components

Production data access uses NumPy memory mapping so the 5.36 GiB image corpus is validated without loading complete arrays into process memory. The reusable ChestMNIST dataset component enforces split names, shapes, data types and label width before exposing samples to training or validation code.

ChestMNIST Fixed Dataset Partitions

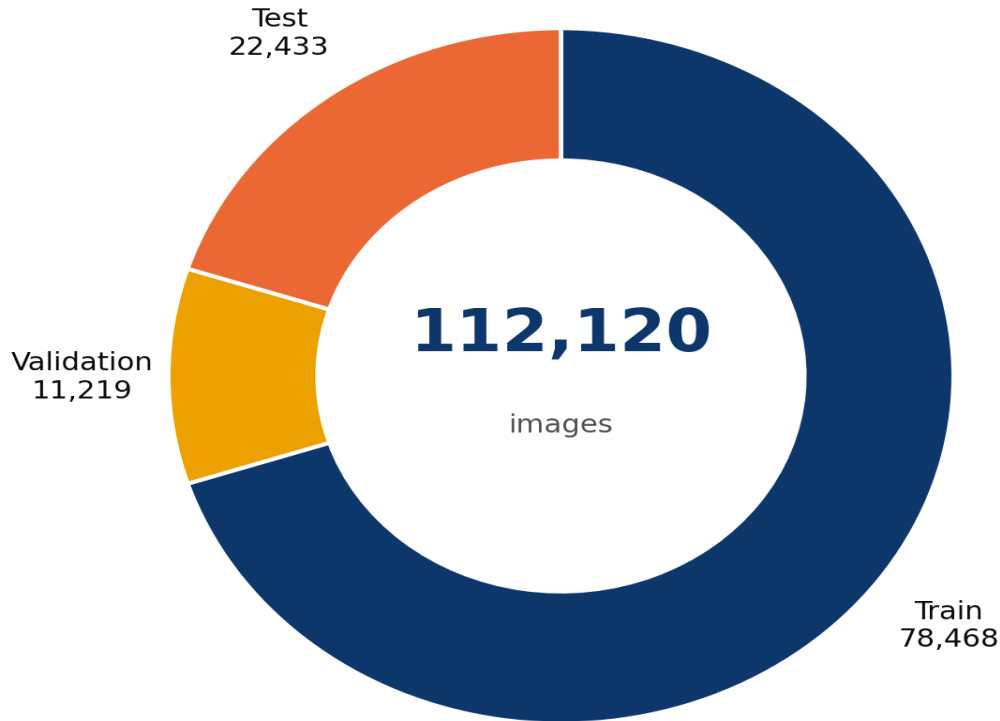


Figure 7. ChestMNIST split composition used by the data-access contract.

Split	Image shape	Label shape	Image dtype	Label dtype	Memmap
train	(78,468, 224, 224)	(78,468, 14)	uint8	uint8	True
val	(11,219, 224, 224)	(11,219, 14)	uint8	uint8	True
test	(22,433, 224, 224)	(22,433, 14)	uint8	uint8	True

8.1 Reusable model factory

`src.models.resnet18_factory` constructs a fourteen-output ResNet-18 under an explicit freezing policy. The small-batch trainer owns the optimizer-facing head parameters while leaving the backbone frozen. A two-sample smoke batch produced shape (2, 14) with 11,183,694 total parameters and 7,182 trainable parameters.

8.2 Training behavior test

8

Batch size

20

Optimization steps

0.7892 →
0.1132

Loss trajectory

85.65%

Relative reduction

The CUDA test validates loss reduction, head updates, frozen backbone invariance and non-creation of a checkpoint. The observed behavior test passed in 9.47 seconds.

9. Test Strategy and Regression Evidence

The test strategy separates deterministic component behavior, ML-specific contracts, adversarial validation, API integration, experimentation policy and deployment packaging. Tests are designed around externally observable contracts rather than implementation line coverage alone.

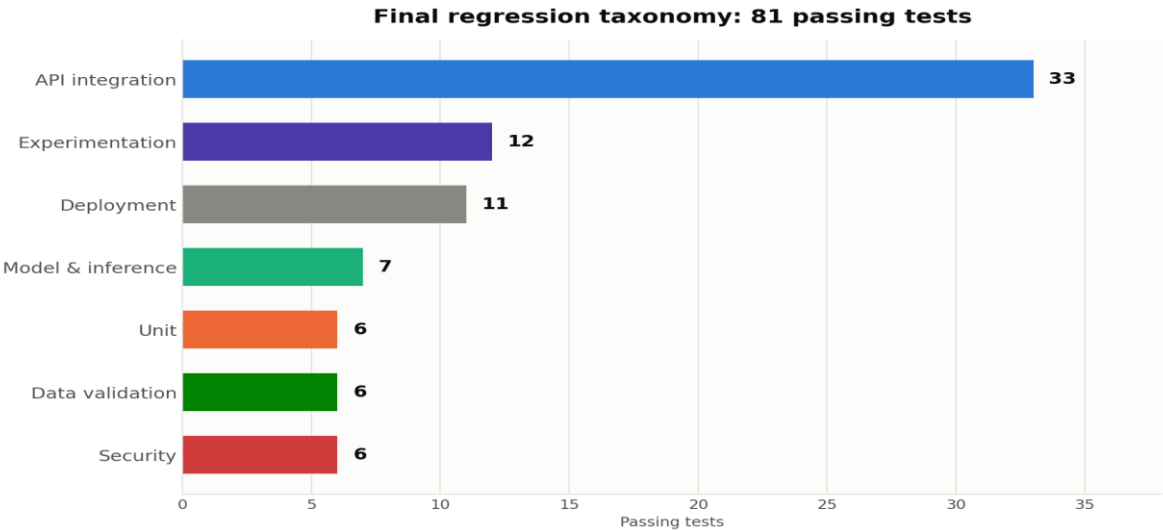


Figure 8. Final regression distribution by engineering concern.

Category	Validated behavior	Passing tests
Unit	Structured log events and sensitive-content exclusion	6
Data validation	Schema, missingness, binary labels, isolation and drift	6
Model and inference	Training loss, frozen backbone, shape, label order and parity	7
Security	Content spoofing, corruption, size, dimensions and controlled errors	6
API integration	Workflows, retrieval, system endpoints and OpenAPI schema	33
Experimentation	Deterministic sampling, isolation, metrics and kill switch	12
Deployment	Docker descriptors, health checks, dependencies and artifact boundaries	11

9.1 Final repository-wide execution

81

Tests passed

1

Warning

71.64 s

Regression time

0

Failures

Black, isort, Ruff and compileall also returned zero in the same final validation boundary. Services were not started and models were not loaded during the repository-wide gate; model behavior was exercised separately by dedicated tests.

10. Data Quality and Drift Controls

Data validation evaluates schema correctness, finite-value completeness, binary-label validity, split-artifact isolation and inter-split label-distribution stability. All measurements operate on the authoritative split artifacts and preserve the image-level multilabel interpretation contract.

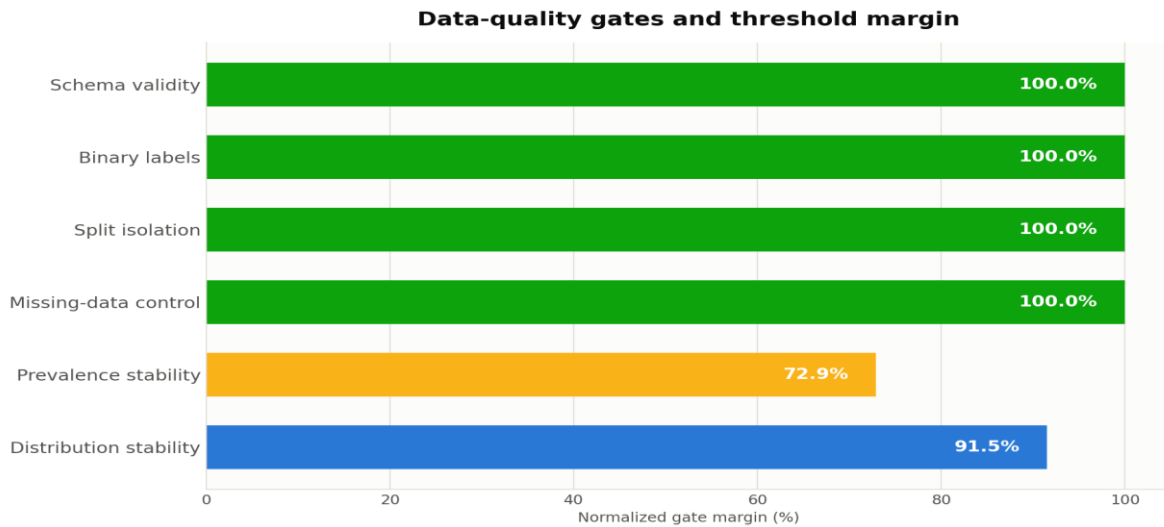


Figure 9. Normalized data-quality gate margins; higher values indicate greater threshold headroom.

Metric	Observed	Threshold	State
Schema-validity rate	1.00000000	≥ 1.000000	PASS
Missing / non-finite rate	0.00000000	≤ 0.000000	PASS
Binary-label-validity rate	1.00000000	≥ 1.000000	PASS
Split-artifact isolation rate	1.00000000	≥ 1.000000	PASS
Maximum prevalence shift	0.00813528	≤ 0.030000	PASS
Maximum Jensen–Shannon divergence	0.00084628	≤ 0.010000	PASS

The largest prevalence shift occurs for atelectasis between validation and test partitions. The maximum Jensen–Shannon divergence also occurs for validation versus test. Both remain materially inside their configured gates.

11. Model Quality and Inference Integrity

Model-quality evidence is read from the frozen test-partition evaluation artifacts for resnet18-chestmnist-v1. The evidence covers 22,433 images and fourteen output labels. Threshold recalibration was not performed, the test inference was not rerun for this report, and the extracted values preserve parity with the recorded MLflow outputs.

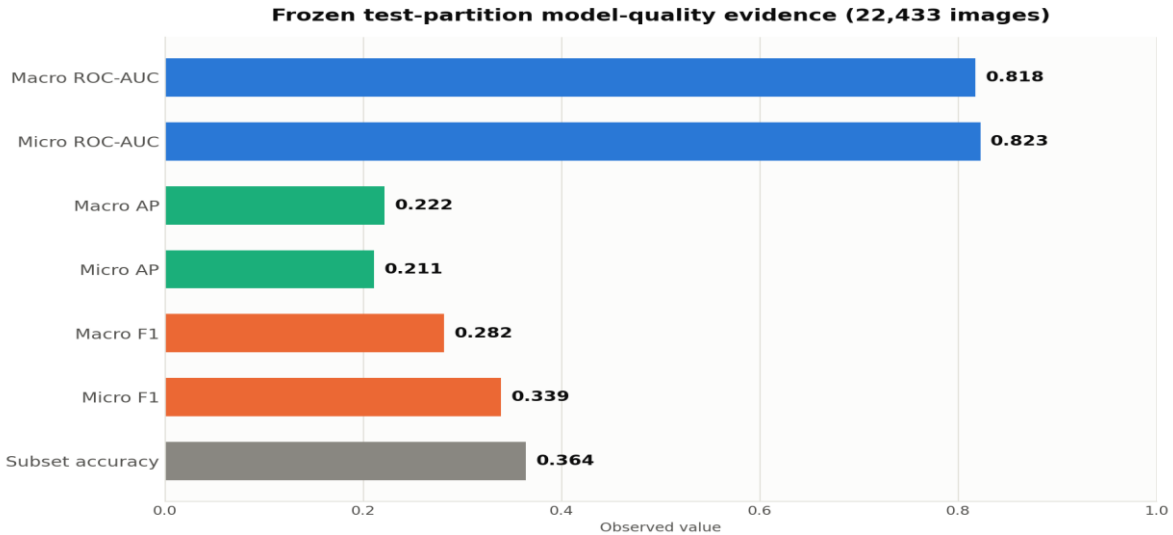


Figure 10. Selected frozen test-partition metrics.

Metric	Macro	Micro
ROC-AUC	0.817548	0.822593
Average precision	0.221623	0.211137
Precision	0.224136	0.261507
Recall	0.415256	0.482132
F1	0.281583	0.339091

Additional multilabel measures: Hamming loss = 0.098799; subset accuracy = 0.364151. ROC-AUC and ranking performance must be interpreted separately from threshold-dependent precision, recall and exact-subset measures.

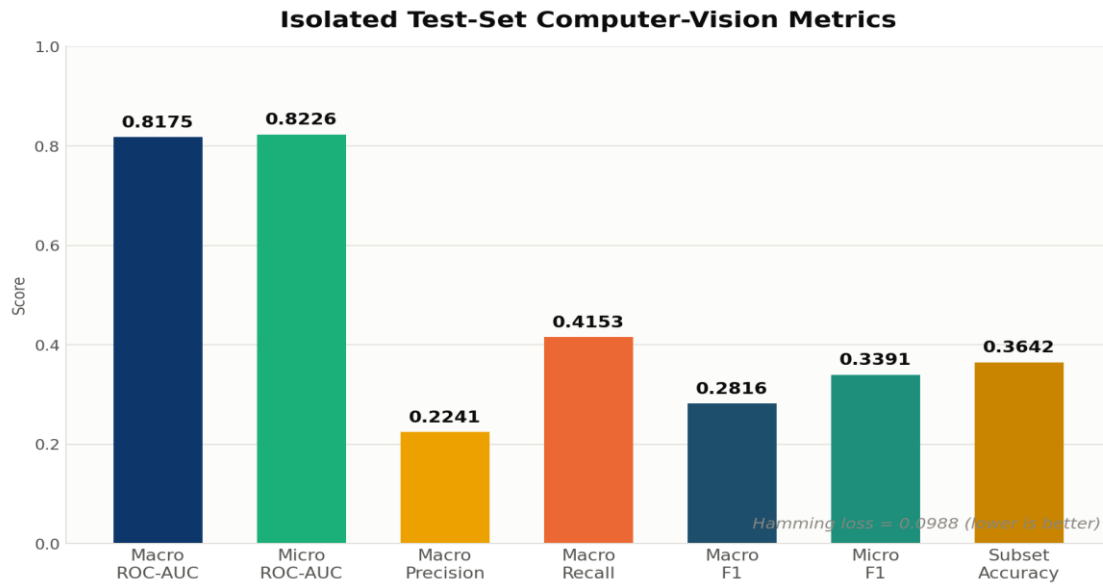


Figure 11. Recorded isolated-test evaluation summary for the frozen model.

11.1 Explainability contract

Grad-CAM is generated for findings whose probabilities cross configured thresholds. The output carries the target finding, heatmap/overlay payload, layer identity and a limitation string. Generation is serialized by a lock because gradient hooks and shared model state are not independently re-entrant.

Grad-CAM Smoke Test: Atelectasis True Positive

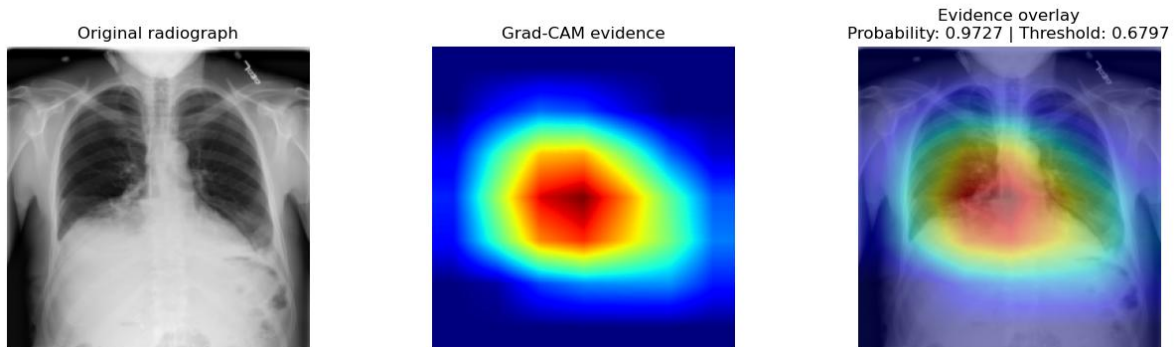


Figure 12. Grad-CAM smoke validation showing the raw radiograph, heatmap and overlay.

12. FastAPI Contract and Application Integration

FastAPI provides the typed boundary around the domain services. Request models forbid unknown fields where appropriate, upload endpoints use content-aware validation, response schemas carry model versions and request metadata, and application-level exception handlers map domain failures to stable API error structures.

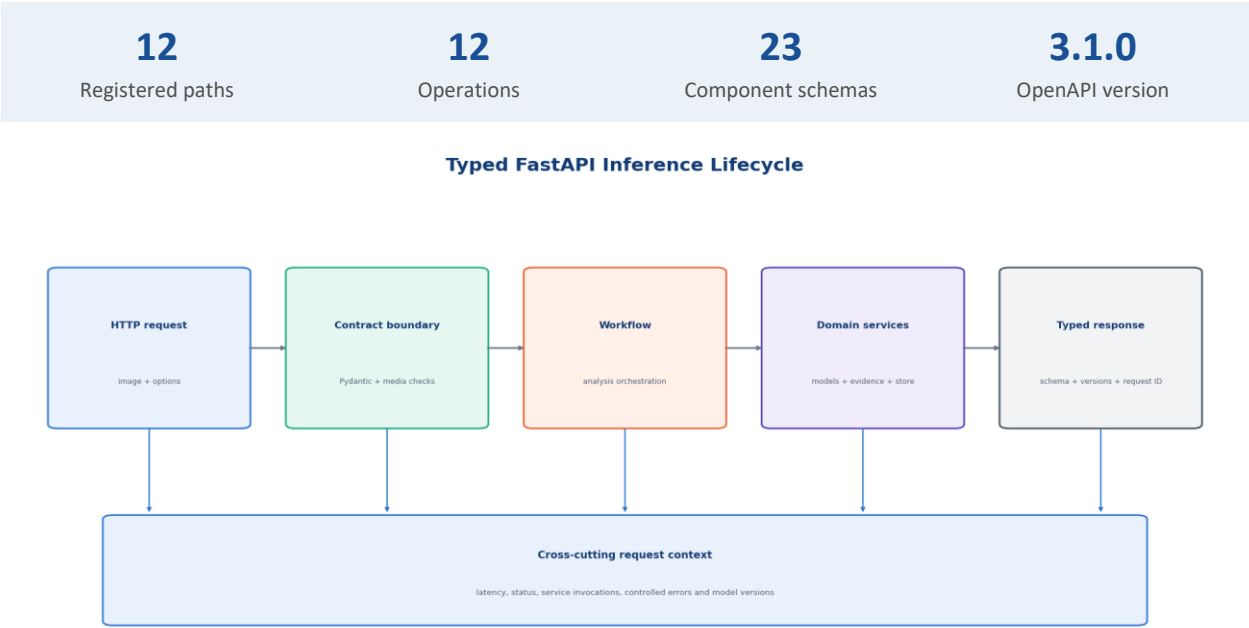


Figure 13. Typed inference lifecycle from request validation through workflow execution, domain services and telemetry.

health		^
GET	/health	Check service readiness
system		^
GET	/api/v1/model/info	Return versioned model information
GET	/api/v1/model/metrics	Return frozen model evaluation metrics
GET	/api/v1/llmops/metrics	Return API operational metrics
analysis		^
POST	/api/v1/image/classify	Classify a chest X-ray image
POST	/api/v1/image/analyze	Classify an image and generate visual evidence
POST	/api/v1/report/generate	Generate a grounded preliminary model report
POST	/api/v1/explanation/generate	Generate a grounded plain-language explanation
POST	/api/v1/question/answer	Answer a question using stored model output
POST	/api/v1/follow-up/recommend	Generate controlled educational follow-up
complete-analysis		^
POST	/api/v1/analyze-complete	Run the complete chest X-ray analysis workflow
GET	/api/v1/predictions/{prediction_id}	Retrieve one stored prediction
Schemas		^

Figure 14. OpenAPI workflow and system endpoints exposed by the service.

12.1 Integration behavior

- Image workflows support classification-only and classification-plus-Grad-CAM execution.

- Language workflows retrieve an existing prediction, serialize its grounded evidence and persist guarded outputs by task type.
- The aggregate workflow executes image analysis, report generation, explanation, optional question answering and follow-up generation under one request context.
- System endpoints publish health, model metadata, model quality and operational metrics independently of prediction execution.

Figure 15. Application interface connected to the typed API client.

Figure 16. Integrated image-analysis response with ordered finding evidence.

13. Security Engineering

The upload boundary distrusts both the filename and declared media type. Validation checks the accepted media-type allowlist, reads bounded content, rejects empty uploads, verifies image decoding and integrity, compares the decoded format with the declared type, and enforces maximum pixel dimensions before inference.

Threat / failure mode	Control	Verified result
Unsupported declared type	Allowlist before decoding	Rejected through typed service error
Empty upload	Non-zero content requirement	Rejected before preprocessing
Oversized payload	Maximum byte boundary	Rejected with received and maximum sizes
Corrupt image	Pillow decode and integrity verification	Rejected without inference
Media spoofing	Declared type versus decoded format comparison	Rejected as mismatch
Decompression / dimension abuse	Maximum width and height contract	Rejected before tensor creation

6

Security tests

0.21 s

Execution time

0

Failures

0

Raw-content log events

13.1 Error-information control

Client details are sanitized before serialization. Unexpected exceptions receive a request identifier and controlled error code while internal stack traces remain server-side. Logging tests separately verify that user-supplied image content and grounded-language inputs are not emitted.

14. Shadow Deployment Strategy

A deterministic policy selects a stable subset of requests for candidate inference while preserving the active model as the only client-visible decision path. The candidate runs asynchronously within the policy boundary, its errors are recorded as experiment outcomes, and neither its output nor its failure can change the active response.

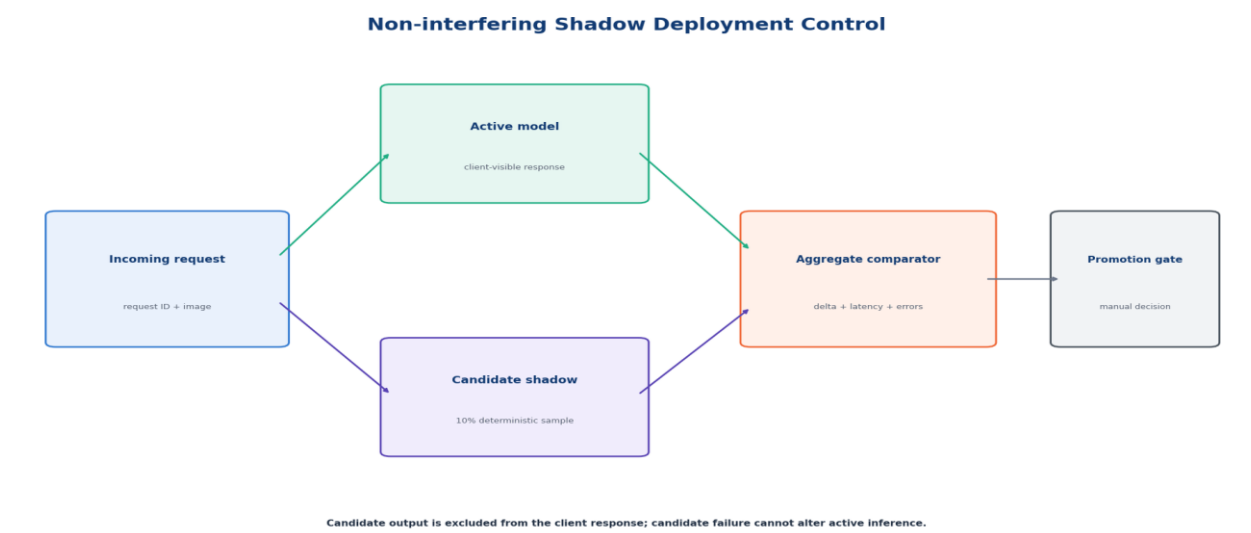


Figure 17. Shadow execution, aggregate comparison and manual promotion boundary.

Control	Configured value
Shadow enabled	True
Configured sampling rate	10.00%
Observed routing experiment	1,039 of 10,000 requests (10.39%)
Candidate output visible to client	False
Candidate failure affects response	False
Minimum comparison volume	1,000 samples
Maximum candidate error rate	0.5%
Maximum decision disagreement	2.0%
Maximum mean probability delta	3.0%
Maximum p95 latency overhead	75 ms

Only aggregate comparison metrics are retained; raw images and generated text are excluded. Promotion remains a manual release decision and the policy exposes a kill switch for immediate deactivation.

15. Packaging, Reproducibility and Deployment Readiness

Deployment descriptors separate the API and UI into independent images. Docker Compose provides both services, health checks and an explicit UI dependency on API readiness. Dataset and model artifacts remain external mounts, preventing large or licensed runtime assets from being embedded in the source package or container build context.

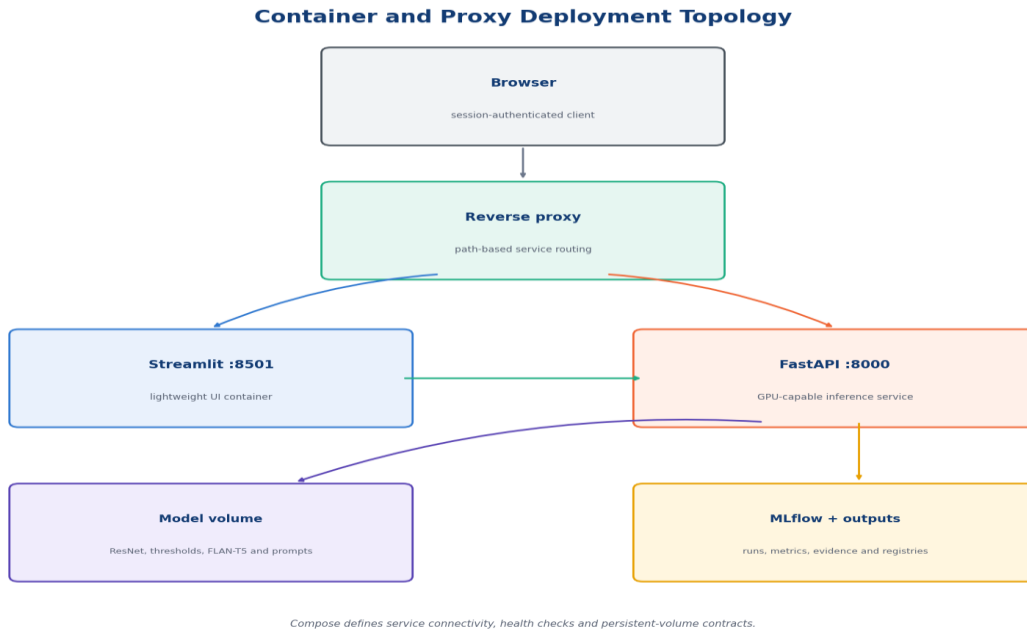


Figure 18. Container and reverse-proxy deployment topology.

Packaging contract	Observed state
Compose services	api, ui
API health check	Present
UI health check	Present
UI dependency on API	Present
API direct dependencies	17 pinned entries
UI direct dependencies	6 pinned entries
Large model/data embedded	No
Packaging contract tests	11 passed
Persistent free space	8.71 GiB
Protected reserve	4.00 GiB

15.1 Reproducibility boundary

requirements.api.txt, requirements.ui.txt and requirements.dev.txt isolate runtime and development dependencies. YAML and JSON files version dataset labels, model thresholds, prompt task prefixes, API response templates, file-system paths and explainability contracts. The package can therefore be reconstructed without relying on notebook-global variables.

The Kubeflow environment did not expose the Docker Compose CLI, so image construction and service startup were intentionally not claimed. Deployment readiness is supported by static descriptor validation and eleven executable packaging-contract tests.

16. Evidence Integrity and Final Readiness

The evidence-integrity manifest establishes a verifiable chain from source files and generated QA artifacts to the final readiness decision. Required JSON artifacts are parsed structurally, text evidence is validated as non-empty, and every in-scope file receives a SHA-256 digest.

36/36 Required evidence 52 Evidence files 80 Repository hashes SHA-256 Integrity algorithm		
Final gate	State	Primary evidence
Repository modularity	PASS	repository_modularity_audit.json
Service-layer contracts	PASS	service_layer_contract_audit.json
Structured logging	PASS	logging_validation.json + logging_pytest.txt
Code quality	PASS	final_engineering_validation.json
Data quality	PASS	data_quality_metrics.json
Model quality	PASS	model_quality_metrics.json
Inference parity	PASS	research_production_parity.json
Research-production comparison	PASS	research_production_comparison.json
Security validation	PASS	security_test_results.json
OpenAPI contract	PASS	openapi_contract_validation.json
Shadow deployment	PASS	shadow_deployment_strategy.json
Packaging and deployment	PASS	packaging_deployment_validation.json
Evidence integrity	PASS	evidence_integrity_manifest.json

Manifest SHA-256: a9a626ea2b27fd39c2af252fd596e77bcd31339d3b39e4ceb5f4d0de9a212c15

17. Technical Limitations and Next Engineering Iterations

The final readiness state demonstrates engineering completeness for the validated environment; it does not eliminate known operational and model boundaries. These limitations are expressed as concrete implementation constraints and measurable next-stage controls.

Current limitation	Engineering consequence	Next controlled iteration
In-memory prediction store	Records do not survive process restart and are local to one replica	Replace with a bounded persistent store using the existing repository contract
Container runtime unavailable in the notebook	Images and inter-service health were not executed in Kubeflow	Run the existing packaging suite plus live smoke tests in CI with Docker/Compose
Single frozen test distribution	Metrics do not establish transportability to new institutions	Introduce external validation and cohort-specific drift dashboards
Threshold-dependent multilabel output	Precision/recall trade-offs depend on the frozen operating points	Version threshold changes and require held-out calibration evidence
Grad-CAM localization is post-hoc	Heatmaps indicate model sensitivity, not causal or pixel-level ground truth	Add expert-reviewed localization evaluation and failure taxonomy
Grounded language depends on serialized findings	Generation quality is bounded by classifier evidence and prompt contracts	Measure groundedness, omission and contradiction on an expanded locked set
Shadow policy is simulated	Promotion gates are defined but not yet fed by production telemetry	Connect aggregate metrics to the release pipeline and retain manual approval

18. Conclusion

The completed system demonstrates a full research-to-production engineering path for a multimodal chest X-ray application. Notebook state was converted into explicit modules, configuration and service contracts; inference parity preserved the frozen model behavior; data, security and API boundaries were made executable; and packaging, experimentation and evidence integrity were validated independently.

The final repository contains 62 validated Python files and twelve test modules. Eighty-one tests pass, Black/isort/Ruff/compileall are clean, all twelve API operations satisfy the OpenAPI contract, and all thirteen final readiness gates pass. The resulting software is reproducible, traceable and structured for controlled deployment and subsequent engineering iteration.

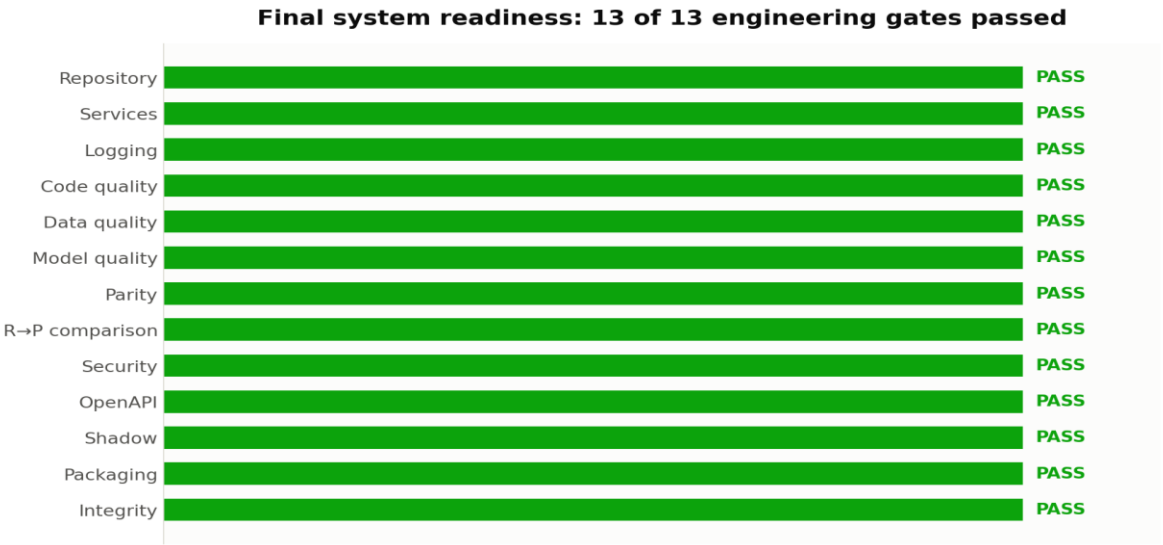


Figure 19. Final engineering validation state: all controlled readiness gates passed.

Appendix A. Endpoint Catalogue

Method	Path	Response schema	Responsibility
POST	/api/v1/analyze-complete	CompleteAnalysisResponse	End-to-end image and optional language workflow
POST	/api/v1/image/classify	ClassificationResponse	Validated image classification
POST	/api/v1/image/analyze	ImageAnalysisResponse	Classification plus visual evidence
POST	/api/v1/report/generate	LanguageGenerationResponse	Grounded report generation
POST	/api/v1/explanation/generate	LanguageGenerationResponse	Grounded explanation generation
POST	/api/v1/question/answer	LanguageGenerationResponse	Grounded question answering
POST	/api/v1/follow-up/recommend	LanguageGenerationResponse	Grounded follow-up generation
GET	/api/v1/predictions/{prediction_id}	StoredPredictionResponse	Prediction retrieval
GET	/api/v1/model/info	ModelInfoResponse	Model and finding contract metadata
GET	/api/v1/model/metrics	ModelMetricsResponse	Frozen quality metrics
GET	/api/v1/llmops/metrics	OperationalMetricsResponse	Request and language-action telemetry
GET	/health	HealthResponse	Component readiness

The OpenAPI schema contains twelve unique operation identifiers and twenty-three component schemas. Ten dedicated contract tests validate schema structure, operation uniqueness and response references.

A.1 HTTP response semantics

Status	Trigger	External contract
200	Successful workflow or system query	Declared response_model schema
400	Decoded content is not a valid image	INVALID_IMAGE
404	Prediction identifier is not present	PREDICTION_NOT_FOUND
413 / 415	Upload exceeds size limit or media type is unsupported	UPLOAD_TOO_LARGE / UNSUPPORTED_MEDIA_TYPE
422	Request-schema validation fails	Controlled validation response
503	Required model component is unavailable	MODEL_NOT_READY
500	Grounding or service execution contract fails	Sanitized controlled error

Appendix B. Notebook and Artifact Traceability

The implementation notebook is the executable engineering narrative; the report consolidates its validated outputs into architectural decisions, contract descriptions and evidence tables. Large datasets and frozen model files remain external to the notebook and report.

Engineering concern	Notebook evidence block	Repository / artifact anchor
Environment and topology	Kernel, filesystem and static AST diagnostics	repository_modularity_audit.json
OOP architecture	Service-layer contract inspection	service_layer_contract_audit.json
Logging	Three implementation audits and behavior tests	logs/*.json; logging_pytest.txt
Code quality	Toolchain, baseline, remediation and final gates	code_quality/before, after and final
Production data/model	Memmap module, model factory and trainer	src/data; src/models; src/training
ML QA	Data, training, inference and frozen model tests	data_quality; metrics; test_results
API/security	OpenAPI contract and upload-security tests	openapi_schema.json; security_test_results.json
Experimentation	Shadow routing and non-interference tests	shadow_deployment_strategy.json
Deployment	Packaging descriptors and tests	docker-compose.yaml; Dockerfiles; packaging evidence
Final readiness	Repository-wide validation and 13-gate decision	system_readiness_gate.json; integrity manifest

Notebook artifact: 89.ipynb · 58 final cells · 0 error-output cells · SHA-256
f0dae0c23ee5ac2fd434862b00ad5d3549e7d97f495c1193d4a339741b2c57dc

Appendix C. Evidence Registry

Evidence family	Key artifacts	Validated property
Architecture	repository_modularity_audit.json; service_layer_contract_audit.json; openapi_schema.json	Topology, public contracts and API schema
Logging	image_validation_logging_implementation.json; computer_vision_logging_implementation.json; language_guardrail_logging_implementation.json	Event coverage and sensitive-content exclusion
Code quality	baseline_code_quality.json; remediation_registry.json; final_code_quality.json	Before/after findings and transactional remediation
Data quality	data_quality_metrics.json; data_validation_pytest.txt	Schema, missingness, label validity and drift
Model quality	model_quality_metrics.json; inference parity artifacts	Frozen metrics, output contract and numerical equivalence
Testing	test_taxonomy.json; full_test_results.json; final_full_pytest.txt	Test coverage and repository-wide regression
Security	security_test_results.json; security_pytest.txt	Adversarial upload boundary
Experimentation	shadow_deployment_strategy.json; shadow_deployment_pytest.txt	Sampling, non-interference and promotion gates
Deployment	packaging_deployment_validation.json; packaging_contract_pytest.txt	Container descriptors, dependencies and artifact isolation
Readiness	final_engineering_validation.json; system_readiness_gate.json; evidence_integrity_manifest.json	Final quality gates and file integrity

Appendix D. Selected Implementation Excerpts

The excerpts below expose the executable boundaries behind the architecture, logging, API and ML-quality evidence. They are drawn from the validated repository and retain the same public contracts exercised by the automated tests.

D.1 Memory-mapped production data access

```
def open_split(self, split: str) -> tuple[np.ndarray, np.ndarray]:
    paths = self.array_paths(split)
    images = np.load(
        paths.images, mmap_mode="r", allow_pickle=False
    )
    labels = np.load(
        paths.labels, mmap_mode="r", allow_pickle=False
    )
    return images, labels
```

Production data access opens persisted arrays without loading the full image corpus into memory.

D.2 Structured validation event

```
logger.info(
    "image_validation_started",
    extra={
        "component": "image_validation",
        "event": "validation_started",
        "media_type": media_type,
        "received_bytes": len(content),
    },
)
```

The event records scalar operational context while excluding filenames and image content.

D.3 Typed inference endpoint

```
@router.post(
    "/image/classify",
    response_model=ClassificationResponse,
    summary="Classify a chest X-ray image",
)
async def classify_image(
    image: UploadFile = File(...),
    services: ServiceContainer = Depends(get_service_container),
) -> ClassificationResponse:
    request_context = RequestContext()
    validated_image = await _validate_upload(
        upload=image,
        services=services,
    )
    execution = await run_in_threadpool(
        services.image_analysis_workflow.classify,
        validated_image=validated_image,
        request_context=request_context,
    )
    return execution.response
```

The route delegates validation and inference to injected services and returns a strict response schema.

D.4 Small-batch training assertion

```
trace = trainer.overfit_batch(
    images,
    targets,
    steps=optimization_steps,
)

assert torch.isfinite(torch.tensor(trace.losses)).all()
assert trace.final_loss < trace.initial_loss
```

The behavioral test requires a finite optimization trace and an observed loss reduction.

References

1. MedMNIST v2. <https://medmnist.com/>
2. PyTorch documentation. <https://pytorch.org/docs/stable/index.html>
3. TorchVision ResNet models. <https://pytorch.org/vision/stable/models/resnet.html>
4. Grad-CAM: Visual Explanations from Deep Networks. <https://arxiv.org/abs/1610.02391>
5. FastAPI documentation. <https://fastapi.tiangolo.com/>
6. Pydantic documentation. <https://docs.pydantic.dev/>
7. pytest documentation. <https://docs.pytest.org/>
8. Black documentation. <https://black.readthedocs.io/>
9. Ruff documentation. <https://docs.astral.sh/ruff/>
10. Docker Compose specification. <https://docs.docker.com/compose/compose-file/>